leetcode 1

最长公共前缀

C

```c
char* longestCommonPrefix(char** strs, int strsSize) {
    char *s0=strs[0];
    for (int i=0;s0[i];i++) {
        for (int j=0;j<strsSize;j++) {
            if (strs[j][i]!=s0[i]) {
                s0[i]='\0';
                return s0;
            }
        }
    }
    return s0;
}
```

两数之和

C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* twoSum(int* nums, int numsSize, int target, int* returnSize) {
    for (int i = 0; ; i++) { // 枚举 i
        for (int j = i + 1; j < numsSize; j++) { // 枚举 i 右边的 j
            if (nums[i] + nums[j] == target) { // 满足要求
                int* ans = malloc(2 * sizeof(int)); // 分配返回结果的内存空间
                *returnSize = 2; // ans 的长度
                ans[0] = i; // 存储第一个数的下标
                ans[1] = j; // 存储第二个数的下标
                return ans; // 返回结果数组的指针
            }
        }
    }
}
```

回文数

C

```c
bool isPalindrome(int x) {
    if (x<0 || (x>0 && x%10==0)) return false;
    else {
        long int t=0,r=x;
        while (r/10!=0) {
            t=t*10+r%10;
            r=r/10;
        }
        t=t*10+r;
        if (t==x) return true;
        else return false;
    }
}
```

正确括号

C

```c
#include <stdlib.h>
#include <stdbool.h>//注意这个库

typedef struct Node {
    char data;
    struct Node* prev;  // 指向栈中前一个节点（即下一个更低的节点）
} Node;

// 入栈：将字符 s 压入由 top 指向的栈顶（top 是 Node**，用于修改栈顶指针）
void inStack(char s, Node** top) {
    Node* newNode = (Node*)malloc(sizeof(Node));//一定要有(Node*)
    if (!newNode) return; // 内存分配失败（实际项目应处理）
    newNode->data = s;
    newNode->prev = *top; // 新节点的 prev 指向原栈顶  prev为指针，故应对应*top同为指针
    *top = newNode;       // 更新栈顶为新节点  指针对应指针
}

// 出栈：弹出栈顶元素，返回其 data；若栈空，返回 '\0'；同时更新 *top
char pop(Node** top) {
    if (!*top) return '\0'; // 栈空
    Node* temp = *top;
    char data = temp->data;
    *top = temp->prev; // 更新栈顶为下一个节点
    free(temp);
```

```
    return data;
}

// 判断括号字符串是否有效
bool isValid(char* s) {
    if (!s) return true; // 空指针视为有效（或可返回 false，按需）
    Node* top = NULL; // 初始栈为空（注意：不再 malloc 哑节点！）
    int i = 0;
    while (s[i] != '\0') {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') {
            inStack(c, &top); // 注意：传 &top（Node**）
        } else if (c == ')' || c == ']' || c == '}') {
            char topChar = pop(&top);// 检查匹配性
            if ((c == ')' && topChar != '(') ||
                (c == ']' && topChar != '[') ||
                (c == '}' && topChar != '{')) {
                return false;
            }
        }// 忽略非括号字符（题目通常只含括号，但健壮性考虑）
        i++;
    }
    return top == NULL;// 最终栈必须为空才合法
}
```

C

```
bool isValid(char* s) {//灵神解法
    int top = 0; // 直接把 s 当作栈！！！
    for (int i = 0; s[i]; i++) {
        char c = s[i];
        if (c == '(') {
            s[top++] = ')'; // 入栈对应的右括号
        } else if (c == '[') {
            s[top++] = ']';
        } else if (c == '{') {
            s[top++] = '}';
        } else if (top == 0 || s[--top] != c) { // c 是右括号
            return false; // 没有左括号，或者左括号类型不对
        }
    }
    return top == 0; // 所有左括号必须匹配完毕
}

```

删除重复元素

C

```c
int removeDuplicates(int* nums, int numsSize) {
    if (numsSize == 0) return 0;
    int slow = 0;  // 慢指针：指向当前唯一元素的最后一个位置
    for (int fast = 1; fast < numsSize; fast++) {
        // 如果发现新元素
        if (nums[fast] != nums[slow]) {
            slow++;  // 慢指针先移动
            nums[slow] = nums[fast];  // 更新为新的唯一元素
        }
    }
    return slow + 1;  // 返回唯一元素的个数
}
int removeDuplicates(int* nums, int numsSize) {
    int k = 1;
    for (int i = 1; i < numsSize; i++) {
        if (nums[i] != nums[i - 1]) { // nums[i] 不是重复项
            nums[k++] = nums[i]; // 保留 nums[i]
        }
    }
    return k;
}
```

最早重复位置

C

```c
int strStr(char* haystack, char* needle) {
    int h_len = strlen(haystack);
    int n_len = strlen(needle);
    if (n_len == 0) {
        return 0;
    }
    if (h_len < n_len) {
        return -1;
    }
    for (int i = 0; i <= h_len - n_len; i++) {
        if (strncmp(haystack + i, needle, n_len) == 0) {//注意str处理方法
            return i;//不引入变量一直使用strlen会导致重复计算时间太长
        }
    }
    return -1;
```

```
}
```

判断两树相等

C

```c
bool isSameTree(struct TreeNode* p, struct TreeNode* q) {
    if (p==NULL && q==NULL) return true;
    else if (p==NULL || q==NULL) return false;
    else if (p->val != q->val) return false;
    else return isSameTree(p->left,q->left)&&isSameTree(p->right,q->right);
}
```

对称二叉树

C

```c
bool isSymmetric(struct TreeNode* root) {
    return isSame(root->left,root->right);
}
bool isSame(struct TreeNode* p, struct TreeNode* q) {
    if (p==NULL || q==NULL) return p==q;
    else return p->val==q->val && isSame(p->left,q->right) && isSame(q->left,p->right);
}
```

异或和

C

```c
int xor_n(int n) {
    switch (n % 4) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
        default: return 0;
    }
}
int xorOperation(int n, int start) {
    int a = start / 2;
    int b = n & start & 1; // 都为奇数才是 1
    return (xor_n(a + n - 1) ^ xor_n(a - 1)) * 2 + b;
```

```
}

int xorOperation(int n, int start) {
    int ans=0,i;
    for (i=0;i<n;i++) {
        ans ^= start+2*i;
    }
    return ans;
}
```

2的幂

C

```
bool isPowerOfTwo(int n) {
    if (n == 1) return true;
    else if (n % 2 == 1 || n <= 0) return false;
    else return isPowerOfTwo(n>>1);
}
bool isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

3的幂

C

```
bool isPowerOfThree(int n) {
    return n > 0 && 1162261467 % n == 0;
}
bool isPowerOfThree(int n) {
    if (n==1) return true;
    else if (n<1 || n%3!=0) return false;
    else return isPowerOfThree(n/3);
}
```

235之积

C

```
bool isUgly(int n) {
    if (n <= 0) {
        return false;
```

```
    }
    while (n % 3 == 0) {
        n /= 3;
    }
    while (n % 5 == 0) {
        n /= 5;
    }
    return (n & (n - 1)) == 0;
}
bool isUgly(int n) {
    if (n==1 || n==2 || n==3 || n==5) return true;
    else if (n<1 || (n%2!=0 && n%3!=0 && n%5!=0)) return false;
    else if (n%2==0) return isUgly(n/2);
    else if (n%3==0) return isUgly(n/3);
    else return isUgly(n/5);
}
```

各位之和

C

```
int addDigits(int num) {
    int sum=0;
    if (num<10) return num;
    else {
        while (num/10!=0) {
            sum+=num%10;
            num/=10;
        }
        return addDigits(sum+num);
    }
}
int addDigits(int num) {
    return (num - 1) % 9 + 1;
}
```

重排数组

C

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* shuffle(int* nums, int numsSize, int n, int* returnSize) {
```

```
    int* ans=malloc(numsSize*sizeof(int));
    for (int i=0;i<n;i++) {
        ans[2*i]=nums[i];
        ans[2*i+1]=nums[n+i];
    }
    *returnSize=numsSize;
    return ans;
}
```

分割字符串求01数

C

```
int maxScore(char* s) {
    int ans=0,i,len=strlen(s);
    for (i=1;i<len;i++) {
        int temp=0;
        for (int j=0;j<len;j++) {
            if (s[j]=='0' && j<i) temp++;
            if (s[j]=='1' && j>=i) temp++;
        }
        if (temp>ans) ans=temp;
    }
    return ans;
}
int maxScore(char* s) {
    int n0=0,n1=0,i,ans=0;
    for (i=0;s[i];i++) {
        if (s[i]=='0') n0++;
        if (s[i]=='1') n1++;
    }
    int temp=n1,n=n0+n1-1;
    for (i=0;i<n;i++) {
        if (i==0 && s[i]=='1') temp--;
        else if (i==0 && s[i]=='0') temp++;
        else if (i!=0 && s[i]=='0') temp++;
        else if (i!=0 && s[i]=='1') temp--;
        if (temp>ans) ans=temp;
    }
    return ans;
}
```

转置矩阵

C

```c
/**
 * Return an array of arrays of size *returnSize.
 * The sizes of the arrays are returned as *returnColumnSizes array.
 * Note: Both returned array and *columnSizes array must be malloced, assume caller calls free().
 */
int** transpose(int** matrix, int matrixSize, int* matrixColSize, int* returnSize, int** returnColumnSizes) {
    int m = matrixSize, n = matrixColSize[0];
    *returnSize = n;
    *returnColumnSizes = malloc(sizeof(int) * n);
    int** ans = malloc(sizeof(int*) * n);
    for (int i = 0; i < n; i++) {
        (*returnColumnSizes)[i] = m;
        ans[i] = malloc(sizeof(int) * m);
    }
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            ans[j][i] = matrix[i][j];
        }
    }
    return ans;
}
```

好三数组！！！

C

```c
#define MIN(a, b) ((b) < (a) ? (b) : (a))
#define MAX(a, b) ((b) > (a) ? (b) : (a))
int countGoodTriplets(int* arr, int arrSize, int a, int b, int c) {
    int mx = 0;
    for (int i = 0; i < arrSize; i++) {
        mx = MAX(mx, arr[i]);
    }//找出数组中最大值
    int* s = calloc(mx + 2, sizeof(int)); // cnt 数组的前缀和（值域前缀和）
    int ans = 0;
    for (int j = 0; j < arrSize; j++) {
        int y = arr[j];
        for (int k = j + 1; k < arrSize; k++) {
            int z = arr[k];
            if (abs(y - z) > b) {
                continue;
```

```
            }
            int l = MAX(MAX(y - a, z - c), 0);
            int r = MIN(MIN(y + a, z + c), mx);
            ans += MAX(s[r + 1] - s[l], 0); // 如果 l > r + 1，s[r + 1] - s[l] 可能是负数
        }
        for (int v = y + 1; v < mx + 2; v++) {
            s[v]++; // 把 y 加到 cnt 数组中，更新所有受到影响的前缀和
        }
    }
    free(s);
    return ans;
}
```

山脉数组的峰顶

C

```
int peakIndexInMountainArray(int* arr, int arrSize) {
    int i=arrSize/2;
    if (arrSize==3) return 1;
    else if (arr[i]>arr[i-1] && arr[i]>arr[i+1]) return i;
    else if (arr[i-1]<arr[i] && arr[i]<arr[i+1]) return peakIndexInMountainArray(arr+i, arrSize-i)+i;
    else return peakIndexInMountainArray(arr, i+1);
}
```

数组窗口内最大平均值

C

```c
double findMaxAverage(int* nums, int numsSize, int k) {
    int i,sum=INT_MIN,temp=0;
    for (i=0;i<numsSize;i++) {
        if (i<k) {
            temp+=nums[i];
        }
        else {
            temp+=(nums[i]-nums[i-k]);
        }
        if (temp>sum && i>=k-1) sum=temp;//注意必须在到达一定程度后再更新ans
    }
    return (double)sum/k;
}
```

字符串窗口内最大元音数

C

```c
int maxVowels(char* s, int k) {
    int ans=0,len=strlen(s);
    for (int i=0;i<k;i++) {
        if (s[i]=='a' || s[i]=='e' || s[i]=='i' || s[i]=='o' || s[i]=='u') ans++;
    }
    int temp=ans;//注意变量位置，确保能够及时更新，如果定义在循环内则一直保持原ans值，没有实时变化窗口内的数量
    for (int i=0;i<len-k;i++) {
        if (s[i]=='a' || s[i]=='e' || s[i]=='i' || s[i]=='o' || s[i]=='u') temp--;
        if (s[i+k]=='a' || s[i+k]=='e' || s[i+k]=='i' || s[i+k]=='o' || s[i+k]=='u') temp++;
        if (temp>ans) ans=temp;
    }
    return ans;
}
```

半径为k数组平均值

C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* getAverages(int* nums, int numsSize, int k, int* returnSize) {
```

```
    int* ans=malloc(numsSize*sizeof(int));
    *returnSize=numsSize;
    long long sum=0;
    for (int i=0;i<numsSize;i++) {
        sum+=nums[i];
        if (i<k || i>=numsSize-k) ans[i]=-1;
        if (i>2*k) sum-=nums[i-2*k-1];
        if (i>=2*k) ans[i-k]=sum/(2*k+1);
    }
    return ans;
}
```

得到k个连续B的最少次数

C

```
int minimumRecolors(char* blocks, int k) {
     int n=strlen(blocks),b=0,max=0;
     for (int i=0;i<n;i++) {
         if (blocks[i]=='B') b++;
         if (i>=k && blocks[i-k]=='B') b--;
         if (i>=k-1 && b>max) max=b;
     }
     return k-max;
}
```

拆炸弹（变为下/前k个数的和）

C

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* decrypt(int* code, int codeSize, int k, int* returnSize) {
    int* ans=malloc(codeSize*sizeof(int));
    *returnSize=codeSize;
    if (k==0) {
        for (int i=0;i<codeSize;i++) {
            ans[i]=0;
        }
    }
    else if (k>0) {
        for (int i=0;i<codeSize;i++) {
            ans[i]=0;
```

```
            for (int j=0;j<k;j++) {
                ans[i]+=code[(i+j+1)%codeSize];
            }
        }
    }
    else {
        for (int i=0;i<codeSize;i++) {
            ans[i]=0;
            for (int j=0;j>k;j--) {
                ans[i]+=code[(i+(codeSize+j-1))%codeSize];
            }
        }
    }
    return ans;
}
```

股票买卖时机

C

```
long long maxProfit(int* prices, int pricesSize, int* strategy, int strategySize, int k) {
    long long sum=0,temp=0;
    for (int i=0;i<pricesSize;i++) {
        sum+=prices[i]*strategy[i];
    }
    temp=sum;
    for (int i=0;i<k;i++) {
        if (i<k/2) temp-=prices[i]*strategy[i];
        else temp+=(prices[i]-prices[i]*strategy[i]);
    }
    if (temp>sum) sum=temp;
    for (int i=k;i<pricesSize;i++) {
        temp+=(prices[i]*(1-strategy[i])+prices[i-k]*strategy[i-k]-prices[i-k/2]);
        if (temp>sum) sum=temp;
    }
    return sum;
}
```

K近邻下标

C

```c
#define MIN(a, b) ((b) < (a) ? (b) : (a))
#define MAX(a, b) ((b) > (a) ? (b) : (a))
int* findKDistantIndices(int* nums, int numsSize, int key, int k, int* returnSize) {
    int* ans = malloc(sizeof(int) * numsSize);
    *returnSize = 0;
    int j = 0;
    for (int i = 0; i < numsSize; i++) {
        if (nums[i] != key) {
            continue;
        }
        j = MAX(j, i - k); // j 至少是 i-k
        while (j <= MIN(i + k, numsSize - 1)) { // j 至多是 i+k，但不能越界
            ans[(*returnSize)++] = j++;
        }
    }
    return ans;
}
```

最短数组

C

```c
int minSubArrayLen(int target, int* nums, int numsSize) {
    int sum = 0, left = 0, ans = numsSize+1 ;
    for ( int right = 0; right < numsSize; right++ ) {
        sum += nums[right] ;
        while ( sum >= target ) {
            ans = right - left + 1 < ans ? right - left + 1 : ans ;
            sum -= nums[left] ;
            left++ ;
        }
    }
    return ans > numsSize ? 0 : ans;
}
```

删去一个元素全为1的最长数组

C

```c
int longestSubarray(int* nums, int numsSize) {
```

```c
    int n = 0, left = 0, ans = 0;
    for ( int right = 0; right < numsSize; right++ ) {
        if ( nums[right] == 0 && n == 0 ) n++ ;
        else if ( nums[right] == 0 && n == 1 ) {
            ans = ans > right - left ? ans : right - left ;
            while ( n == 1 ) {
                if ( nums[left] == 0 ) n--;
                left++ ;
            }
        }
    }
    return ans ;
}//不成立的程序
//此处为正确程序
int longestSubarray(int* nums, int numsSize) {
    int n = 0, left = 0, ans = 0;
    for ( int right = 0; right < numsSize; right++ ) {
        n += 1 - nums[right];
        while (n > 1) {
            n -= 1 - nums[left];
            left++;
        }
        ans = ans > right - left ? ans : right - left ;
    }
    return ans ;
}
```

使数组两最值小于k倍的最小移除数量

```c
```

```c
int cmp(const void* a, const void* b) {
    return *(int*)a - *(int*)b;
}//此为从小到大，反之为从大到小
int minRemoval(int* nums, int numsSize, int k) {//思路上就是不从外向答案压，而是让两指针往答案方向扩展
    qsort(nums, numsSize, sizeof(int), cmp);//标准的qsort使用方式：数组，数组大小，数组元素大小，标准的函数->就是说要背过那个cmp函数
    int ans=numsSize, right=0;
    for (int left=0; left<numsSize; left++) {
        while (right<numsSize && (long long)nums[right]<=(long long)nums[left]*k) right++;//在一个left下让right合理地外扩
        ans=numsSize-right+left<ans?numsSize-right+left:ans;//重视使用三元符
    }
    return ans;
}
```

904水果成篮

C

```c
int totalFruit(int* fruits, int fruitsSize) {
    int ans=0,left=0,a=fruits[left],b=-1;
    for (int right=0; right<fruitsSize; right++) {
        if (fruits[right]!=a && b==-1) b=fruits[right];
        else if (fruits[right]!=a && fruits[right]!=b) {
            left=right-1;
            while (fruits[left-1]==fruits[right-1]) left--;
            a=fruits[left];
            b=fruits[right];
            continue;
        }
        ans=right-left+1>ans?right-left+1:ans;
    }
    return ans;
}
```

1004翻去k各0后连续1的最大数量

C

```c
int longestOnes(int* nums, int numsSize, int k) {
    int ans=0, left=0, sum=0;
    for (int right=0; right<numsSize; right++) {
        sum+=1-nums[right];//就是0的个数
```

```
        while (sum>k) {//在0的数量依旧大于k时
            sum-=1-nums[left];//在不断减少0的数量，这样可以简便判断
            left++;
        }
        ans=right-left+1>ans?right-left+1:ans;
    }
    return ans;
}
```

2024改变k个答案得到的连续相同答案长度

C

```c
int maxConsecutiveAnswers(char* answerKey, int k) {
    int ans=0, left=0, len=strlen(answerKey), sum=0;
    for (int right=0; right<len; right++) {
        sum+=answerKey[right]=='T'?1:0;
        while (sum>k) {
            sum-=answerKey[left]=='T'?1:0;
            left++;
        }
        ans=right-left+1>ans?right-left+1:ans;
    }
    sum=0;
    left=0;
    for (int right=0; right<len; right++) {
        sum+=answerKey[right]=='F'?1:0;
        while (sum>k) {
            sum-= (answerKey[left]=='F'?1:0);
            left++;
        }
        ans=right-left+1>ans?right-left+1:ans;
    }
    return ans;
}
```

2730找出最长的只有一组连续相同数字的字符串

C

```c
int longestSemiRepetitiveSubstring(char* s) {
    int n=0,len=strlen(s), left=0,ans=0;
    if (len==1) return 1;
    for (int right=0;right<len-1;right++) {
```

```
        if (s[right]==s[right+1]) {
            n++;
        }
        if (n>1) {
            while (s[left]!=s[left+1]) {
                left++;
            }
            left++;
            n--;
        }
        ans=right-left+2>ans?right-left+2:ans;
    }
    return ans;
}
```

658K个最接近数字

C

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* findClosestElements(int* arr, int arrSize, int k, int x, int*
returnSize) {
    *returnSize = k;  // 修正1：返回数组大小为k

    int left = 0, right = arrSize - 1;
    int* ans = malloc(k * sizeof(int));

    // 修正2：使用正确的双指针缩小范围逻辑
    while (right - left + 1 > k) {
        // 比较左右指针元素与x的距离
        int leftDist = abs(arr[left] - x);
        int rightDist = abs(arr[right] - x);

        // 根据题目规则移动指针
        if (rightDist < leftDist) {
            // 右元素更接近，保留右指针区域，左指针右移
            left++;
        } else if (leftDist < rightDist) {
            // 左元素更接近，保留左指针区域，右指针左移
            right--;
        } else {
            // 距离相等时，取较小的元素（即左边的元素）
            right--;
        }
```

```
    }

    // 修正3：正确填充结果数组
    for (int i = 0; i < k; i++) {
        ans[i] = arr[left + i];
    }

    return ans;
}
```

1750删除两端相同后最小长度

C

```
int minimumLength(char * s) {
    int n = strlen(s);
    int left = 0, right = n - 1;
    while (left < right && s[left] == s[right]) {
        char c = s[left];
        while (left <= right && s[left] == c) {
            left++;
        }
        while (left <= right && s[right] == c) {
            right--;
        }
    }
    return right - left + 1;
}
```

167两数之和 有序数组

C

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* twoSum(int* numbers, int numbersSize, int target, int* returnSize) {
    int left=0,right=numbersSize-1;
    while (1) {
        int s=numbers[left]+numbers[right];
        if (s==target) {
            *returnSize=2;
            int* ans=malloc(2*sizeof(int));
            ans[0]=left+1;
```

```
            ans[1]=right+1;
            return ans;
        }
        else s>target?right--:left++;
    }
}
```

485 01串中最大连续1的长度

C

```c
int findMaxConsecutiveOnes(int* nums, int numsSize) {
    int ans=0,cnt=0;
    for (int i=0;i<numsSize;i++) {
        if (nums[i]==1) {
            cnt++;
            ans=ans>cnt?ans:cnt;
        }
        else cnt=0;
    }
    return ans;
}//可以不用计量左右端值的right和left，可以使用cnt
```

2110股票平滑下降日期（其期间的任意日期区间）

C

```c
long long getDescentPeriods(int* prices, int pricesSize) {
    long long ans=0;
    int cnt=0;
    for (int right=0;right<pricesSize;right++) {
        if (right>0 && prices[right]+1==prices[right-1]) {
            cnt++;
        }
        else {
            cnt=1;
        }
        ans+=cnt;
    }
    return ans;
}
```

1855下标对中的最大距离

C

```c
//给两个非递增数组，需要给出满足nums1[i]<=nums2[j] && i<=j的数组的最大长度，即j-i的最大值
int maxDistance(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int ans=0,i=0;
    for (int j=0;j<nums2Size;j++) {
        while (i<=j && i<nums1Size && nums1[i]>nums2[j]) i++;//注意此处并不是都写上就行了，这些判断是有顺序的
        if (i<=j && i<nums1Size && nums1[i]<=nums2[j]) ans=j-i>ans?j-i:ans;
    }
    return ans;
}
```

925长按键入

C

```c
bool isLongPressedName(char* name, char* typed) {
    int i = 0, j = 0;
    int lenn = strlen(name), lent = strlen(typed);
    while (i < lenn && j < lent) {// 如果当前字符匹配，两个指针都前进
        if (name[i] == typed[j]) {
            i++;
            j++;
        } // 如果typed当前字符与前一个相同（长按情况），只移动typed指针
        else if (j > 0 && typed[j] == typed[j-1]) {
            j++;
        }// 否则不匹配
        else {
            return false;
        }
    }// 处理typed末尾的长按字符
    while (j < lent && typed[j] == typed[j-1]) {
        j++;
    }// 只有当name和typed都遍历完才算成功
    return i == lenn && j == lent;
}
```

838推多米诺

C

```c
char* pushDominoes(char* dominoes) {
    int left=0,right=0,len=strlen(dominoes),dot=0;
    for (int i=0;i<len;i++) {
        dot+=dominoes[i]=='.'?0:1;
    }
    if (dot==0 || dot==len) return dominoes;
    while (left<len) {
        while (dominoes[left]!='.' && left+1<len) left++;
        right=left;
        while (right+1<len && dominoes[right+1]=='.') right++;
        if (right<len-1 && left==0 && dominoes[right+1]=='L') {
            for (int i=right;i>=0;i--) dominoes[i]='L';
        }
        else if (left>0 && right==len-1 && dominoes[left-1]=='R') {
            for (int i=left;i<len;i++) dominoes[i]='R';
        }
        else if (left>0 && right<len-1 && dominoes[left-1]==dominoes[right+1]) {
            for (int i=left;i<=right;i++) dominoes[i]=dominoes[left-1];
        }
        else if (left>0 && right<len-1 && dominoes[left-1]=='R' && dominoes[right+1]=='L') {
            for (int i=left,j=right;i<j;i++,j--) {
                dominoes[i]=dominoes[left-1];
                dominoes[j]=dominoes[right+1];
            }
        }
        left=right+1;
    }
    return dominoes;
}

char* pushDominoes(char* s) {
    int n = strlen(s);
    int pre = -1; // 上一个 L 或 R 的位置
    for (int i = 0; i <= n; i++) {
        char ch = i < n ? s[i] : 'R';
        if (ch == '.') {
            continue;
        }
        if (ch == (pre < 0 ? 'L' : s[pre])) { // L...L 或 R...R
            memset(s + pre + 1, ch, i - pre - 1); // 全变成 s[i]
        } else if (ch == 'L') { // R...L
            int half = (i - pre - 1) / 2;
            memset(s + pre + 1, 'R', half); // 前一半变 R
```

```
            memset(s + i - half, 'L', half); // 后一半变 L
        }
        pre = i;
    }
    return s;
}

```

34重复元素始末位置

C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int lowerBound(int* nums, int numsSize, int target) {
    int left = 0, right = numsSize - 1; // 移除外部 mid 初始化
    while (left <= right) {
        int mid = left + (right - left) / 2; // 修正 mid 计算，避免溢出
        if (nums[mid] >= target) { // 移除不必要的 mid>0 条件
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return left; // 返回第一个大于或等于 target 的索引
}

int* searchRange(int* nums, int numsSize, int target, int* returnSize) {
    int *ans = malloc(2 * sizeof(int));
    ans[0] = -1;
    ans[1] = -1;
    *returnSize = 2;

    int start = lowerBound(nums, numsSize, target);
    // 添加边界检查：如果 start 越界或 nums[start] 不等于 target，则未找到
    if (start == numsSize || nums[start] != target) {
        return ans;
    } else {
        int end = lowerBound(nums, numsSize, target + 1) - 1; // 计算结束位置
        ans[0] = start;
        ans[1] = end;
    }
    return ans;
}
```

35搜索插入位置

C

```c
int searchInsert(int* nums, int numsSize, int target) {
    int left=0,right=numsSize-1;
    while (left<=right) {
        int mid=left+(right-left)/2;
        if (nums[mid]==target) return mid;
        else if (nums[mid]>target) right=mid-1;
        else if (nums[mid]<target) left=mid+1;
```

```
    }
    return left;
}
```

744比目标数字大的最小字母

C

```c
char nextGreatestLetter(char* letters, int lettersSize, char target) {
    if (target>=letters[lettersSize-1]) return letters[0];
    int left=0,right=lettersSize-1;
    while (left<right) {
        int mid=left+(right-left)/2;
        if (letters[mid]>target) right=mid;
        else left=mid+1;
    }
    return letters[left];
}
```

2529正负整数最大数量

C

```c
int maximumCount(int* nums, int numsSize) {
    if (nums[0]==0 && nums[numsSize-1]==0) return 0;
    int left=0,right=numsSize-1;
    while (left<right) {
        int mid=left+(right-left)/2;
        if (nums[mid]>=0) right=mid;
        else left=mid+1;
    }
    int neg=nums[left]>=0?left:left+1;
    left=0;
    right=numsSize-1;
    while (left<right) {
        int mid=left+(right-left)/2;
        if (nums[mid]>0) right=mid;
        else left=mid+1;
    }
    return (numsSize-left)>neg?(numsSize-left):neg;
}
```

1385数组间距离值（arr1中令所有arr2全不在起+-d区间内的数量）

C

```c
int cmp(const void* a,const void* b) {return *(int*) a-*(int*) b;}
int lower(int *arr2, int arr2Size, int target) {
    int left=0,right=arr2Size-1;
    while (left<right) {
        int mid=left+(right-left)/2;
        if (arr2[mid]>=target) right=mid;
        else left=mid+1;
    }
    return arr2[left]>=target?left:arr2Size;
}
int upper(int* arr2, int arr2Size, int target) {
    int left=0,right=arr2Size-1;
    while (left<right) {
        int mid=left+(right-left)/2;
        if (arr2[mid]>target) right=mid;
        else left=mid+1;
    }
    return arr2[left]>target?left:arr2Size;
}
int findTheDistanceValue(int* arr1, int arr1Size, int* arr2, int arr2Size,
int d) {
    qsort(arr1, arr1Size, sizeof(int),cmp);
    qsort(arr2, arr2Size, sizeof(int),cmp);
    int ans=0;
    for (int i=0;i<arr1Size;i++) {
        int left=lower(arr2, arr2Size, arr1[i]-d);
        int right=upper(arr2, arr2Size, arr1[i]+d);
        ans+=right-left==0?1:0;
    }
    return ans;
}
```

2389和有限的最长子序列

C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int cmp(const void* a,const void* b) {return *(int*)a-*(int*)b;}
int* answerQueries(int* nums, int numsSize, int* queries, int queriesSize,
int* returnSize) {
    *returnSize=queriesSize;
```

```
    qsort(nums, numsSize, sizeof(int), cmp);
    int sum[numsSize+1];
    for (int i=0;i<=numsSize;i++) {
        if (i==0) sum[i]=0;
        else sum[i]=sum[i-1]+nums[i-1];
    }
    for (int i=0;i<queriesSize;i++) {
        int left=0,right=numsSize;
        while (left<right) {
            int mid=left+(right-left)/2;
            if (queries[i]>=sum[mid]) left=mid+1;
            else right=mid;
        }
        queries[i]=queries[i]>=sum[numsSize]?numsSize:left-1;
    }
    return queries;
}
```

2300咒语与代码配对对数

C

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int cmp(const void* a, const void* b) {return *(int*)a-*(int*)b;}

int* successfulPairs(int* spells, int spellsSize, int* potions, int potionsSize, long long success, int* returnSize) {
    *returnSize=spellsSize;

    if (potionsSize == 0) {
        int* result = malloc(spellsSize * sizeof(int));
        for (int i = 0; i < spellsSize; i++) result[i] = 0;
        return result;
    }

    qsort(potions, potionsSize, sizeof(int), cmp);

    for (int i=0;i<spellsSize;i++) {
        long long target = success / spells[i];
        if (success % spells[i] != 0) target++;

        int left=0, right=potionsSize-1;

        // 使用 while (left < right) 的二分查找
```

```
        while (left < right) {
            int mid = left + (right-left)/2;
            if (potions[mid] >= target) {
                right = mid;  // 满足条件，尝试更小的mid
            } else {
                left = mid + 1;  // 不满足条件，尝试更大的mid
            }
        }

        // 循环结束后，left 指向第一个可能的满足条件的位置
        // 但仍需检查是否真的满足条件
        int result = 0;
        if (potions[left] >= target) {
            result = potionsSize - left;
        }
        spells[i] = result;
    }
    return spells;
}

```

C

```c
//2563数对数目
int cmp(const void* a, const void* b) {return *(int*)a-*(int*)b;}
long long countFairPairs(int* nums, int numsSize, int lower, int upper) {
    qsort(nums, numsSize, sizeof(int), cmp);//对nums进行排序便于二分
    long long ans=0;
    for (int i=numsSize-1;i>0;i--) {//从大到小逐个计数
        int l_left=0,u_left=0,l_right=i-1,u_right=i-1;
        int l_find=lower-nums[i],u_find=upper-nums[i];
        while (l_left<l_right) {
            int l_mid=l_left+(l_right-l_left)/2;
            if (nums[l_mid]>=l_find) l_right=l_mid;
            else l_left=l_mid+1;
        }
        if (nums[l_left]<l_find) continue;
        while (u_left<u_right) {
            int u_mid=u_left+(u_right-u_left+1)/2;//此处需要+1
            if (nums[u_mid]<=u_find) u_left=u_mid;
            else u_left=u_mid-1;
        }
        if (nums[u_left]<=u_find) ans+=u_right-l_left+1;
    }
    return ans;
}
```

```c
int cmp(const void* a, const void* b) {return *(int*)a-*(int*)b;}

long long countFairPairs(int* nums, int numsSize, int lower, int upper) {
    qsort(nums, numsSize, sizeof(int), cmp);
    long long ans = 0;

    for (int i = numsSize-1; i > 0; i--) {
        long long l_find = (long long)lower - nums[i];
        long long u_find = (long long)upper - nums[i];

        int l_left = 0, l_right = i-1;
        int u_left = 0, u_right = i-1;

        // 查找第一个 >= l_find 的位置
        while (l_left < l_right) {
            int l_mid = l_left + (l_right - l_left) / 2;
            if (nums[l_mid] >= l_find) {
                l_right = l_mid;
            } else {
                l_left = l_mid + 1;
            }
        }

        // 检查左边界是否满足条件
        if (nums[l_left] < l_find) {
            continue; // 没有满足条件的左边界
        }

        // 查找最后一个 <= u_find 的位置
        while (u_left < u_right) {
            int u_mid = u_left + (u_right - u_left + 1) / 2; // 注意这里 +1
            if (nums[u_mid] <= u_find) {
                u_left = u_mid;
            } else {
                u_right = u_mid - 1;
            }
        }

        // 检查右边界是否满足条件
        if (nums[u_left] <= u_find) {
            ans += (u_left - l_left + 1);
        }
    }

    return ans;
}

```

您的第一个程序有两个关键问题导致超时：

1. 第二个二分查找循环逻辑错误（导致死循环）

第一个程序中的第二个二分查找：

```c
while (u_left<u_right) {
    int u_mid=u_left+(u_right-u_left+1)/2;//此处需要+1
    if (nums[u_mid]<=u_find) u_left=u_mid;
    else u_left=u_mid-1;  // 这里应该更新 u_right 而不是 u_left
}
```

问题：

- 当 `nums[u_mid] > u_find` 时，您将 `u_left` 设为 `u_mid-1`
- 但循环条件是 `while (u_left < u_right)`，这意味着如果 `u_left` 减少，区间可能不会正确收缩
- 这可能导致死循环，因为 `u_left` 和 `u_right` 的更新不协调

正确做法：应该更新 `u_right` 而不是 `u_left`：

```c
while (u_left<u_right) {
    int u_mid=u_left+(u_right-u_left+1)/2;
    if (nums[u_mid]<=u_find) u_left=u_mid;
    else u_right=u_mid-1;  // 这里应该更新 u_right
}
```

2. 最终计数使用了错误的变量

第一个程序的最后一行：

```c
if (nums[u_left]<=u_find) ans+=u_right-l_left+1;
```

问题：

- 您使用了 `u_right` 来计算数量

- 但在第二个二分查找中，`u_right` 的值没有更新，仍然是初始值 `i-1`
- 这会导致计数错误，应该使用 `u_left`：

```c
if (nums[u_left]<=u_find) ans+=u_left-l_left+1;  // 使用 u_left
```

为什么这两个问题导致超时?

1. **死循环**：在第二个二分查找中，由于更新逻辑错误，循环可能永远不会满足终止条件 `u_left >= u_right`，导致无限循环。
2. **错误的更新策略**：在二分查找中，必须确保每次迭代都能缩小区间。您的代码中，当 `nums[u_mid] > u_find` 时，将 `u_left` 设为 `u_mid-1`，但此时 `u_right` 保持不变。如果 `u_left` 小于 `u_right`，下一轮循环可能再次计算相同的 `u_mid`，导致无限循环。

第二个程序正确的原因:

第二个程序的第二个二分查找使用了标准的二分查找模式：

- 当条件满足时 (`nums[u_mid] <= u_find`)，更新左边界：`u_left = u_mid`
- 当条件不满足时，更新右边界：`u_right = u_mid - 1`

这样确保了：

1. 每次循环都会缩小区间
2. 不会产生死循环
3. 最终 `u_left` 指向最后一个满足条件的元素位置

1283不超过阈值的最小除数

```c
int cmp(const void* a, const void* b) {return *(int*)a-*(int*)b;}
int smallestDivisor(int* nums, int numsSize, int threshold) {
    qsort(nums, numsSize, sizeof(int), cmp);
    long long sum=numsSize;
    for (int i=0;i<numsSize;i++) {sum+=nums[i];}
    long long left=1,right=sum;
    while (left<right) {
        long long mid=left+(right-left)/2;
        long long result=0;
        for (int i=0;i<numsSize;i++) {
```

```
            result+=nums[i]%mid==0?nums[i]/mid:nums[i]/mid+1;
        }
        if (result<=threshold) right=mid;
        else left=mid+1;
    }
    return left;
}
```

2178旅途次数最短时间

C

```c
long long minimumTime(int* time, int timeSize, int totalTrips) {
    int min_time=INT_MAX;
    for (int i=0;i<timeSize;i++) {
        min_time=time[i]<min_time?time[i]:min_time;
    }
    long long left=1,right=(long long)totalTrips*min_time;
    while (left<right) {
        long long mid=left+(right-left)/2;
        long long result=0;
        for (int i=0;i<timeSize;i++) {
            result+=mid/time[i];
        }
        if (result>=totalTrips) right=mid;
        else left=mid+1;
    }
    return left;
}
```

3824见效数组使为非正的最小k值（使减小k的次数不大于k平方）

C

```c
int minimumK(int* nums, int numsSize) {
    long long sum=0;
    for (int i=0;i<numsSize;i++) {
        sum+=nums[i]>0?nums[i]:0;
    }
    long long left=1,right=(long long)cbrt(sum)*cbrt(sum)*2;
    while (left<right) {
        long long mid=left+(right-left)/2;
        long long result=0;
        for (int i=0;i<numsSize;i++) {
```

```
            result+=(long long)nums[i]>0?(nums[i]%mid==0?
nums[i]/mid:nums[i]/mid+1):0;
        }
        if (result>(long long)mid*mid) left=mid+1;
        else right=mid;
    }
    return left;
}
```

1011D天内送完包裹的最小运载能力

C

```c
int shipWithinDays(int* weights, int weightsSize, int days) {
    int max=0,total=0;
    for (int i=0;i<weightsSize;i++) {
        max=weights[i]>max?weights[i]:max;
        total+=weights[i];
    }
    long left=max,right=total;
    while (left<right) {
        int mid=left+(right-left)/2;//此处的mid讨论的是运载能力
        int count=1,temp=0;//此处count从1开始
        for (int i=0;i<weightsSize;i++) {
            if (temp+weights[i]<=mid) temp+=weights[i];
            else {
                temp=weights[i];
                count++;
            }
        }
        if (count>days) left=mid+1;//注意这里的逻辑
        else right=mid;
    }
    return left;
}
```

875时间内吃香蕉

C

```c
int shipWithinDays(int* weights, int weightsSize, int days) {
    int max=0,total=0;
    for (int i=0;i<weightsSize;i++) {
        max=weights[i]>max?weights[i]:max;
```

```
        total+=weights[i];
    }
    long left=max,right=total;
    while (left<right) {
        int mid=left+(right-left)/2;//此处的mid讨论的是运载能力
        int count=1,temp=0;//此处count从1开始
        for (int i=0;i<weightsSize;i++) {
            if (temp+weights[i]<=mid) temp+=weights[i];
            else {
                temp=weights[i];
                count++;
            }
        }
        if (count>days) left=mid+1;//注意这里的逻辑
        else right=mid;
    }
    return left;
}
```

3856移除尾部原因字母

C

```c
char* trimTrailingVowels(char* s) {
    int len = strlen(s);

    // 从后往前找到第一个非元音字符的位置
    int i = len - 1;
    while (i >= 0 && strchr("aeiou", s[i]) != NULL) {
        i--;
    }

    // 计算新长度
    int new_len = i + 1;

    // 分配内存并复制
    char* ans = malloc(new_len + 1);
    strncpy(ans, s, new_len);
    ans[new_len] = '\0';

    return ans;
}

```

C

```c
//3296!!!
long long minNumberOfSeconds(int mountainHeight, int* workerTimes, int workerTimesSize) {
    long long max=0;
    for (int i=0;i<workerTimesSize;i++) {
        max=workerTimes[i]>max?workerTimes[i]:max;
    }
    long long size=(long long)max*mountainHeight*(mountainHeight+1)/2+1;
    long long left=1,right=size;
    while (left<right) {
        long long mid=left+(right-left)/2;
        long long result=0;
        for (int i=0;i<workerTimesSize;i++) {
            long double tmp = 1.0L + 8.0L * mid / workerTimes[i];
            long long h = (sqrtl(tmp) - 1.0L) / 2.0L;

            // 浮点数误差微调
            if (workerTimes[i] * (h + 1) * (h + 2) / 2 <= mid) {
                h++;
            } else if (workerTimes[i] * h * (h + 1) / 2 > mid) {
                h--;
            }
        }
        if (result>=mountainHeight) right=mid;
        else left=mid+1;
    }
    return left;
}
#define EPS 1e-7

long long minNumberOfSeconds(int mountainHeight, int* workerTimes, int workerTimesSize) {
    int maxWorkerTimes = 0;
    for (int i = 0; i < workerTimesSize; i++) {
        if (workerTimes[i] > maxWorkerTimes) {
            maxWorkerTimes = workerTimes[i];
        }
    }

    long long l = 1;
    long long r = (long long)maxWorkerTimes * mountainHeight * (mountainHeight + 1) / 2;
    long long ans = 0;

    while (l <= r) {
```

```
        long long mid = (l + r) / 2;
        long long cnt = 0;
        for (int i = 0; i < workerTimesSize; i++) {
            long long work = mid / workerTimes[i];
            // 求最大的 k 满足 1+2+...+k <= work
            long long k = (long long)((-1.0 + sqrt(1 + work * 8)) / 2 + EPS);
            cnt += k;
        }

        if (cnt >= mountainHeight) {
            ans = mid;
            r = mid - 1;
        } else {
            l = mid + 1;
        }
    }

    return ans;
}

```

3639变为活跃的最小时间

C

```c
int minTime(char* s, int* order, int orderSize, int k) {
    int total=orderSize*(orderSize+1)/2;//所有子串数量
    if (total<k) return -1;//如果所有子串数量尚且不足k，则必然错误
    // 使用动态分配的二维数组进行模拟
    char** arr = (char**)malloc(orderSize * sizeof(char*));
    for (int i = 0; i < orderSize; i++) {
        arr[i] = (char*)malloc((orderSize + 1) * sizeof(char));  // +1 为字符串结束符
        if (i == 0) {
            strncpy(arr[i], s, orderSize);
            arr[i][orderSize] = '\0';  // 添加字符串结束符
            arr[i][order[i]] = '*';
        } else {
            strncpy(arr[i], arr[i-1], orderSize);
            arr[i][orderSize] = '\0';  // 添加字符串结束符
            arr[i][order[i]] = '*';
        }
    }//通过O(n)操作，递推模拟所有操作
    int left=0,right=orderSize-1;
    while (left<right) {
```

```
        int mid=left+(right-left)/2;
        int result=total,count=0;//使用count计算已有的连续字母数量
        for (int i=0;i<orderSize;i++) {
            if (arr[mid][i]!='*') count++;
            if (arr[mid][i]=='*' || i==orderSize-1) {
                result-=count*(count+1)/2;//通过不断减去不含*的子串数量，反向得到每一个的结果
                count=0;
            }
        }
        if (result>=k) right=mid;//结果多，说明切分过细，需要减少
        else left=mid+1;//数量小于等于，说明切分不足，需要增加
    }// 释放动态分配的内存
    for (int i = 0; i < orderSize; i++) {
        free(arr[i]);
    }
    free(arr);
    return left;
}

int minTime(char* s, int* order, int orderSize, int k) {
    long long total = (long long) orderSize * (orderSize + 1) / 2;  // 所有子串数量
    if (total < k) return -1;  // 如果所有子串数量尚且不足k，则必然错误
    int left = 0, right = orderSize - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        // 动态生成第mid步的状态
        char* current = (char*)malloc((orderSize + 1) * sizeof(char));
        if (!current) return -1;  // 内存分配失败
        // 从原始字符串开始
        strncpy(current, s, orderSize);
        current[orderSize] = '\0';
        // 应用前mid+1步操作（因为mid是从0开始计数的，所以是0到mid）
        for (int i = 0; i <= mid; i++) {
            current[order[i]] = '*';
        }
        // 计算有效子串数
       long long result = total, count = 0;
        for (int i = 0; i < orderSize; i++) {
            if (current[i] != '*') {
                count++;
            }
            if (current[i] == '*' || i == orderSize - 1) {
                result -= (long long) count * (count + 1) / 2;
                count = 0;
            }
        }
```

```
        // 释放当前状态的内存
        free(current);
        if (result >= k) {
            right = mid;  // 结果多，说明切分过细，需要减少
        } else {
            left = mid + 1;  // 数量小于等于，说明切分不足，需要增加
        }
    }
    return left;
}
```

475供暖器

C

```
int cmp(const void* a, const void* b) {return *(int*)a-*(int*)b;}
int findRadius(int* houses, int housesSize, int* heaters, int heatersSize)
{
    //思路是通过双指针遍历，找到房屋与供暖器的最大距离
    qsort(houses, housesSize, sizeof(int), cmp);
    qsort(heaters, heatersSize, sizeof(int),cmp);
    int ans=0,i=0,j=0;
    while (i<housesSize) {
        int current=ans;//记录在此之前的最大值
        ans=abs(houses[i]-heaters[j])>current?abs(houses[i]-
heaters[j]):current;//对于一个i而言需要先赋初值
        while (j<heatersSize-1 && abs(houses[i]-
heaters[j])>=abs(houses[i]-heaters[j+1])) {
            j++;
            ans=abs(houses[i]-heaters[j])>current?abs(houses[i]-
heaters[j]):current;
        }
        i++;
    }
    return ans;
}
```

275至少有h个值大于h

C

```
int hIndex(int* citations, int citationsSize) {
    int left=0,right=citationsSize;
    while (left<right) {
```

```
        int mid=left+(right-left+1)/2;
        if (citations[citationsSize-mid]<mid) right=mid-1;
        else left=mid;
    }
    return left;
}
```

2226分得最多糖果数

C

```c
int maximumCandies(int* candies, int candiesSize, long long k) {
    int size=0;
    long long sum=0;
    for (int i=0;i<candiesSize;i++) {
        size=candies[i]>size?candies[i]:size;
        sum+=candies[i];
    }
    if (sum<k) return 0;
    int left=1,right=size;
    while (left<right) {
        int mid=left+(right-left+1)/2;//!!!!!!
        long long count=0;
        for (int i=0;i<candiesSize;i++) {
            count+=(long long)candies[i]/mid;
        }
        if (count>=k) left=mid;//!!!!!!
        else right=mid-1;//!!!!!!
    }
    return left;
}
```

一道洛谷题

C

```c
#include <stdio.h>
#include <ctype.h>

typedef long long ll;

/* 快速读入整数 */
int readInt() {
    int x = 0, f = 1;
```

```
    char ch = getchar();
    while (!isdigit(ch)) {
        if (ch == '-') f = -1;
        ch = getchar();
    }
    while (isdigit(ch)) {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

/* 快速读入长整型 */
ll readLongLong() {
    ll x = 0, f = 1;
    char ch = getchar();
    while (!isdigit(ch)) {
        if (ch == '-') f = -1;
        ch = getchar();
    }
    while (isdigit(ch)) {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

int main() {
    int T = readInt();
    while (T--) {
        int n = readInt();
        ll cur = readLongLong();  // 当前第一个灵珠的值
        ll sum = 0;
        for (int i = 2; i <= n; i++) {
            ll x = readLongLong(); // 当前第二个灵珠的值
            if (x > 0) {
                sum += x;          // 收走第二个灵珠
            } else {
                cur = x;           // 销毁第一个灵珠，第二个成为新的第一个
            }
        }
        printf("%lld\n", sum);
    }
    return 0;
}

```



题目 | 参考题解 | 讨论区 (67) | 提交记录

MC0405 · 计算灵力值

难度：黄金 时间限制：3秒 占用内存：256 M 收藏 报错

唐僧师徒行至一座荒山，忽然狂风大作，天空瞬间阴云密布。悟空警觉地握紧金箍棒，低声道：“师父，妖气冲天，怕是白骨精又来了！”

果然，不远处一片白雾翻腾，白骨精变幻身形，施展妖法，召唤出一列神秘的魂骨灵珠。灵珠共有 n 颗，并按顺序从1到 n 编号，每颗上都刻着一个整数 a_1，a_2，…，a_n，其中蕴含着不同的灵力。

老妖阴笑道：“此阵乃我精心布下，若想破阵，你们需从中夺取足够的灵力，否则便永远留在此地！”

小码哥挺身而出，仔细研究这些灵珠，发现可以进行如下操作：

1.若选择编号为偶数的灵珠，可将其收走，并将灵珠上的数值加到灵力值中，然后灵珠会消失。

2.若选择编号为奇数的灵珠，可将其销毁，但灵力值不变。

3.每当销毁或收走一颗灵珠，剩下的灵珠会按照从左到右的顺序重新从1开始编号。

现在，小码哥的初始灵力值为 0，他可以进行任意次数操作，但只有获得最高的灵力值，才能助师徒四人成功破阵！

悟空皱眉道：“这妖法好阴险，得想个最佳策略才行。小码哥，快算算最多的灵力是多少吧！”

格式

输入格式： 第一行一个整数 $T(1 \le T \le 10^6)$，表示测试数据组数。
对于每组测试数据：
第一行一个整数 $n(1 \le n \le 10^6)$。
第二行 n 个整数 $a_1 \sim a_n(-10^9 \le a_i \le 10^9)$。
数据保证所有组的 n 的和不超过 10^6。

输出格式： 对于每组测试数据，输出一行一个整数，表示答案。

样例 1

3143无重复标签点的最大正方形中的最大点数

C

```c
int maxPointsInsideSquare(int** points, int pointsSize, int* pointsColSize, char* s) {
    //构建val数组，用于记录每个位置点需要多大的正方形才会被覆盖
    int* val=malloc(sizeof(int)*pointsSize);
    for (int i=0;i<pointsSize;i++) {
        val[i]=-1;
        val[i]=abs(points[i][0])<abs(points[i][1])?abs(points[i][1]):abs(points[i][0]);
    }
    //构建alp数组，其[0]记录最近位置的val值，[1]记录第二近的一个
    int alp[26][2]={0};
    for (int i=0;i<pointsSize;i++) {
        int t = s[i] - 'a';
```

```
        int v = val[i];
        if (alp[t][0] == 0) {
            alp[t][0] = v;
        } else if (v < alp[t][0]) {
            alp[t][1] = alp[t][0];
            alp[t][0] = v;
        } else if (alp[t][1] == 0 || v < alp[t][1]) {
            alp[t][1] = v;
        }
    }
    //用size求出的是最大正方形半边长+1
    int size=INT_MAX;
    for (int i=0;i<26;i++) {
        if (alp[i][1]!=0) size=alp[i][1]<size?alp[i][1]:size;
    }
    //最后用ans记录有多少个点符合要求
    int ans=0;
    for (int i=0;i<pointsSize;i++) {
        if (val[i]<size) ans++;
    }
    free(val);
    return size==INT_MAX?pointsSize:ans;
}
//时间复杂度O(3*n)，空间复杂度O(n)
```

378有序矩阵中总体第k小的数

C

```
int count(int** matrix, int matrixSize, int mid) {
    int num=0;
    for (int i=0;i<matrixSize;i++) {//对于每一行矩阵进行二分查找，找出该行内小于等于这个数的数量
        if (matrix[i][0]>mid) break;//如果第一个数就大了，就可以直接跳出
        int left=0,right=matrixSize-1;
        while (left<=right) {
            int middle=left+(right-left)/2;
            if (matrix[i][middle]<=mid) left=middle+1;
            else right=middle-1;
        }
        num+=left;
    }
    return num;
}
int kthSmallest(int** matrix, int matrixSize, int* matrixColSize, int k) {
    int left=matrix[0][0],right=matrix[matrixSize-1][matrixSize-1];
```

```
    //在这个矩阵范围内二分查找
    while (left<right) {
        int mid=left+(right-left)/2;//这个mid是查找的元素
        int num=count(matrix,matrixSize,mid);//计算小于等于这个数的数字数量
        if (num<k) left=mid+1;
        else right=mid;
    }
    return left;
}
```

162二分寻找峰值

C

```
int findPeakElement(int* nums, int numsSize) {
    int left=-1,right=numsSize-1;//使用left=-1可以保证右侧有值
    while (left+1<right) {
        int mid=left+(right-left)/ 2;
        if (nums[mid]>nums[mid+1]) right=mid;
        else left=mid;
    }
    return right;
}
```

122买卖股票的最大收益（DP）

C

```
int maxProfit(int* prices, int pricesSize) {
    int dp0=0,dp1=-prices[0];
    for (int i=1;i<pricesSize;i++) {
        int newdp0=dp0>dp1+prices[i]?dp0:dp1+prices[i];
        int newdp1=dp1>dp0-prices[i]?dp1:dp0-prices[i];
        dp0=newdp0;
        dp1=newdp1;
    }
    return dp0;
}
//此题使用动态规划十分简便
//思想就是，某一天的收益，只由前一天的情况决定。对于任意前一天的确定收益，都可以反推出当天的情况，具体如下
//首先明确，每一天均处于两种情况之一：手中有和没有股票
//对于某一天，如果手中没有股票，那收益等于前一天没有股票的收益或前一天有股票但是这一天卖出。而后取最大值即可
```

```
15 //同理，如果手中有股票，那收益等于前一天也有股票的收益或前一天没有股票但是今天买入。而后也是取最大值
```

45跳跃游戏（从后向前贪心）

C

```c
int jump(int* nums, int numsSize) {
    int step=0,end=numsSize-1;
    while (end!=0) {
        int temp=end;
        for (int i=end;i>=0;i--) {
            if (end-i<=nums[i]) temp=i;
        }
        end=temp;
        step++;
    }
    return step;
}
```

322兑换零钱的最小数目（DP）

C

```c
int cmp(const void* a,const void*b) {return *(int*)a-*(int*)b;}
int coinChange(int* coins, int coinsSize, int amount) {
    qsort(coins, coinsSize, sizeof(int), cmp);//对硬币数组进行排序
    if (amount<coins[0]) return amount==0?0:-1;//排除不可能完成的情况，同时注意amount=0的情况返回0
    int* ans=malloc(sizeof(int)*(amount+1));//建立动态规划的空间
    for (int i=0;i<=amount;i++) {//对于从0开始的每一个数进行处理
        ans[i]=i==0?0:amount+1;//注意是去最小，那么初始化一个最大的值
        for (int j=0;j<coinsSize;j++) {
            if (coins[j]>i) break;
            //到达一个数的最小步数只由它之前的数的步数决定，那么只需要找出这个数之前的数中可以通过coins到达的当中步数最少的
            else ans[i]=ans[i]>ans[i-coins[j]]+1?ans[i-coins[j]]+1:ans[i];
        }
    }
    int result=ans[amount];
    free(ans);
    return result>amount?-1:result;//返回时注意区分结果究竟是否可行
}
```

198不相邻数之和的最大值（DP）

C

```c
bool wordBreak(char* s, char** wordDict, int wordDictSize) {
    int left=0,len=strlen(s);
    for (int right=0;right<len;right++) {
        char temp[right-left+1];
        strncpy(temp,s+left,right-left+1);
        for (int j=0;j<wordDictSize;j++) {
            if (strcmp(wordDict[j],temp)==0) {
                if (right==len-1) return true;
                else {
                    left=right+1;
                }
            }
            else {
                if (right==len-1) return false;
            }
        }
    }
    return false;
}
```

139拆分单词（DP）

C

```c
bool wordBreak(char* s, char** wordDict, int wordDictSize) {
    int* len=malloc(sizeof(int)*wordDictSize);
    for (int i=0;i<wordDictSize;i++) {
        len[i]=strlen(wordDict[i]);
    }
    bool* ans=malloc(sizeof(int)*strlen(s));
    for (int i=0;s[i];i++) {//对于字符串s中的每一个字母
        ans[i]=false;
        for (int j=0;j<wordDictSize;j++) {
            if (i+1<len[j]) continue;
            else if (i+1==len[j] || ans[i-len[j]]==true) {
                char temp[len[j]];
                strncpy(temp,s+i-len[j]+1,len[j]);
                if (strcmp(temp, wordDict[j])==0) ans[i]=true;
            }
            else continue;
        }
```

```
    }
    bool result=ans[wordDictSize-1];
    free(len);
    free(ans);
    return result;
}
```

80原地删除数组中超过两次的部分

C

```c
int removeDuplicates(int* nums, int numsSize) {
    int count=1,j=0,length=1;//j用来计数修改后的
    if (numsSize==1) return length;
    for (int i=1;i<numsSize;i++) {//i用来计数遍历原数组的
        if (nums[i]==nums[i-1]) count++;
        else count=1;
        if (count<3) {
            nums[j]=nums[i];
            length++;
            j++;
        }
    }
    return length;
}
```

238除自身以外的数组乘积（分为左右分别计算）

C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* productExceptSelf(int* nums, int numsSize, int* returnSize) {
    *returnSize=numsSize;
    int* copy_l=malloc(sizeof(int)*numsSize);
    int* copy_r=malloc(sizeof(int)*numsSize);
    int left=1,right=1;
    for (int i=0;i<numsSize;i++) {
        copy_l[i]=left;
        copy_r[i]=right;
        left*=nums[i];
        right*=nums[numsSize-1-i];
    }
```

```
    for (int i=0;i<numsSize;i++) {
        nums[i]=copy_l[i]*copy_r[i];
    }
    free(copy_r);
    free(copy_l);
    return nums;
}
```

740删除前后相邻值并获取相应点数

C

```c
int cmp(const void* a, const void* b) {return *(int*)a - *(int*)b;}
int deleteAndEarn(int* nums, int numsSize) {
    if (numsSize == 0) return 0;
    qsort(nums, numsSize, sizeof(int), cmp);  // 首先进行排序
    // 统计不同数字的个数
    int number = 1;
    for (int i = 1; i < numsSize; i++) {
        if (nums[i] != nums[i-1]) number++;
    }
    // 创建数组存储不同数字及其出现次数
    int* values = malloc(sizeof(int) * number);   // 存储不同的数字
    int* counts = malloc(sizeof(int) * number);   // 存储每个数字的出现次数
    int index = 0;
    // 统计每个数字的出现次数
    counts[0] = 1;
    values[0] = nums[0];
    for (int i = 1; i < numsSize; i++) {
        if (nums[i] == nums[i-1]) counts[index]++;
        else {
            index++;
            values[index] = nums[i];
            counts[index] = 1;
        }
    }
    // 动态规划计算最大点数
    int* dp = malloc(sizeof(int) * number);
    for (int i = 0; i < number; i++) {
        int earn = values[i] * counts[i];  // 选择当前数字可获得的总点数
        if (i == 0) {
            dp[i] = earn;
        } else if (i == 1) {
            if (values[i] == values[i-1] + 1)
                // 如果当前数字与前一个数字相邻，则不能同时选择
                dp[i] = (earn > dp[i-1]) ? earn : dp[i-1];
```

```
            else {
                // 如果不相邻，可以同时选择
                dp[i] = dp[i-1] + earn;
            }
        } else {
            if (values[i] == values[i-1] + 1) dp[i] = (dp[i-1] > dp[i-2] + earn) ? dp[i-1] : dp[i-2] + earn;
            else dp[i] = dp[i-1] + earn;
        }
    }
    int result = dp[number-1];
    // 释放内存
    free(values);
    free(counts);
    free(dp);
    return result;
}
```